

PPO：近端策略优化

我们已经学习了马尔科夫决策过程，也就是 $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ ，其中包括状态、动作、转移概率、奖励函数和折扣因子。

策略记为 $\pi(a|s)$ ，如果策略带有参数，比如神经网络参数 θ ，就可以写成 $\pi_\theta(a|s)$ ，即策略不是一个固定表格，而是由一个神经网络表示。

输入状态 s ，输出每个动作的概率 $\pi_\theta(a|s)$ ，例如在一个离散动作环境中，神经网络输出 $\pi_\theta(a_1|s) = 0.2; \pi_\theta(a_2|s) = 0.5; \pi_\theta(a_3|s) = 0.3$ ，智能体就按照这个概率分布选择动作。目标就是调整参数 θ ，让智能体获得更高的长期回报。

智能体与环境交互，会产生一条轨迹：

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots)$$

这条轨迹的回报是：

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_{t+1}$$

策略 π_θ 的目标是最大化期望回报：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

也就是说：

$$\theta^* = \arg \max_{\theta} J(\theta)$$

它说明 PPO 的本质并不是直接学习价值函数，而是直接优化策略参数。

我们想让 $J(\theta)$ 变大。按照深度学习的思路，我们希望计算 $\nabla_{\theta} J(\theta)$ ，然后用梯度上升 $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$ ，在普通监督学习中，损失函数通常可以直接对模型输出求导。例如 $L(\theta) = |f_{\theta}(x) - y|^2$ ，但是在强化学习中，目标是 $J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)]$ ，轨迹 τ 本身是由策略 π_{θ} 采样出来的。也就是说， θ 不仅影响动作概率，还影响后续状态、后续奖励和整条轨迹。所以我们要先推导策略梯度。

一条轨迹的概率可以写成 $P_{\theta}(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$ ，其中 $\rho_0(s_0)$ 是初始状态分布。 $\pi_{\theta}(a_t | s_t)$ 是策略选择动作的概率。 $P(s_{t+1} | s_t, a_t)$ 是环境状态转移概率。

于是目标函数可以写成 $J(\theta) = \sum_{\tau} P_{\theta}(\tau) R(\tau)$ ，现在对 θ 求梯度：

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \sum_{\tau} P_{\theta}(\tau) R(\tau)$$

因为奖励 $R(\tau)$ 本身不直接对 θ 求导，所以 $\nabla_{\theta} J(\theta) = \sum_{\tau} \nabla_{\theta} P_{\theta}(\tau) R(\tau)$

接下来使用一个常见技巧：

$$\nabla_{\theta} P_{\theta}(\tau) = P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau)$$

[证明:

因为:

$$\nabla_{\theta} \log P_{\theta}(\tau) = \frac{\nabla_{\theta} P_{\theta}(\tau)}{P_{\theta}(\tau)}$$

所以:

$$\nabla_{\theta} P_{\theta}(\tau) = P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau)$$

代入可得:

$$\nabla_{\theta} J(\theta) = \sum_{\tau} P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau) R(\tau)$$

写成期望形式:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log P_{\theta}(\tau) R(\tau)]$$

现在继续展开:

$$P_{\theta}(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

取对数:

$$\log P_{\theta}(\tau) = \log \rho_0(s_0) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | s_t) + \sum_{t=0}^{T-1} \log P(s_{t+1} | s_t, a_t)$$

对 θ 求梯度。

其中 $\rho_0(s_0)$ 与 θ 无关。环境转移概率 $P(s_{t+1} | s_t, a_t)$ 也与 θ 无关。真正与 θ 有关的只有策略: $\pi_{\theta}(a_t | s_t)$, 所以 $\nabla_{\theta} \log P_{\theta}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$, 代入得到:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

如果一条轨迹的回报高, 那么这条轨迹中出现过的动作概率应该被提高。如果一条轨迹的回报低, 那么这条轨迹中出现过的动作概率应该被降低。

刚才的式子是:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

但是对于时刻 t 的动作 a_t , 它不应该对过去已经发生的奖励负责。所以更合理的做法是使用从当前时刻开始的回报:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

于是策略梯度可以写成:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right]$$

现在有一个问题。

假设某个状态下, 所有动作的回报都比较高。

那某个动作获得了 $G_t = 10$, 是否一定说明它很好?

不一定。如果这个状态下随便采取一个动作都能获得 12，那么 10 反而是差的。

所以我们不应该只看 G_t ，而应该看这个动作比当前状态的平均水平好多少。这个量叫优势函数 $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$ ，其中 $Q^\pi(s_t, a_t)$ 表示在状态 s_t 下采取动作 a_t 后的价值。 $V^\pi(s_t)$ 表示状态 s_t 的平均价值。

如果 $A^\pi(s_t, a_t) > 0$ ，说明这个动作比平均动作更好，应该提高概率。

如果 $A^\pi(s_t, a_t) < 0$ ，说明这个动作比平均动作更差，应该降低概率。

因此策略梯度可以写成 $\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) A^\pi(s_t, a_t) \right]$

这就是 Actor-Critic 方法的核心思想之一。

Actor 是策略网络 $\pi_\theta(a | s)$ ，Critic 是价值网络 $V_\phi(s)$ ，Actor 决定怎么行动，Critic 评价这个行动好不好。

现在我们已经有了策略梯度，看起来可以直接更新 $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$ ，但是普通策略梯度有一个严重问题：每次更新策略之后，新的策略可能和旧的策略差别太大。强化学习的数据是通过旧策略采样出来的。

假设旧策略是 $\pi_{\theta_{\text{old}}}$ ，它采样出一批数据 (s_t, a_t, r_{t+1}) ，如果我们用这批数据训练时，把策略一下子改得太多，新的策略 π_θ 就可能已经不适合这批旧数据了。结果是训练不稳定，回报突然下降，甚至完全崩掉。

PPO 的核心目标不是简单地让策略更新，而是让策略可以进步，但不要离旧策略太远。所以 PPO 里面的关键词是“近端”，新策略要在旧策略附近进行优化。

现在我们遇到一个问题。数据是旧策略 $\pi_{\theta_{\text{old}}}$ 采样的，但是 we 想优化新策略 π_θ ，如果直接用旧数据估计新策略目标，就需要做修正。这个修正项叫重要性采样比率：

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

这个比率表示同一个动作 a_t ，新策略给它的概率相对于旧策略变大了还是变小了。

如果 $r_t(\theta) > 1$ 说明新策略比旧策略更倾向于选择这个动作。

如果 $r_t(\theta) < 1$ 说明新策略比旧策略更不倾向于选择这个动作。

如果 $r_t(\theta) = 1$ 说明新旧策略对这个动作的概率相同。

于是 PPO 的基础优化目标可以写成 $L^{PG}(\theta) = \mathbb{E}_t[r_t(\theta) A_t]$ ，其中 A_t 是优势函数估计。这个式子的意思是：

如果 $A_t > 0$ ，说明动作好，那么希望提高这个动作概率，也就是让 $r_t(\theta)$ 变大。

如果 $A_t < 0$ ，说明动作差，那么希望降低这个动作概率，也就是让 $r_t(\theta)$ 变小。

但是如果只用这个目标，策略仍然可能更新过大。

PPO 不允许 $r_t(\theta)$ 无限变大或无限变小。它规定一个范围 $[1 - \epsilon, 1 + \epsilon]$ ，其中 ϵ 通常取 0.1 或者 0.2。

然后定义裁剪函数 $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$ ，意思是：

如果 $r_t(\theta) > 1 + \epsilon$ ，就把它裁到 $1 + \epsilon$ ；如果 $r_t(\theta) < 1 - \epsilon$ ，就把它裁到 $1 - \epsilon$ ，如

果它本来就在范围内，就保持不变。PPO 的裁剪目标函数是

$$L^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

所以 PPO 的本质是限制新旧策略之间的变化幅度。

在 PPO 之前，有一个重要算法叫 TRPO，也就是 Trust Region Policy Optimization。最大化策略收益，同时限制新旧策略之间的 KL 散度。可以写成 $\max_{\theta} \mathbb{E}_t[r_t(\theta)A_t]$ ，约束条件是 $\mathbb{E}_t[D_{KL}(\pi_{\theta_{\text{old}}}(\cdot | s_t) | \pi_{\theta}(\cdot | s_t))] \leq \delta$ ，这个思想很好，但是实现复杂。因为它是一个带约束优化问题。

PPO 的做法是：不用复杂的二阶优化和严格 KL 约束，而是用一个简单的裁剪目标函数近似实现“不要离旧策略太远”。

实际训练 PPO 时，通常不只训练策略网络，还会训练价值网络。

所以 PPO 的总目标一般包括三部分：

第一部分是策略损失，也就是刚才讲的裁剪目标。

第二部分是价值函数损失，用来训练 Critic。

第三部分是熵奖励，用来鼓励探索。

策略目标就是 PPO 的核心裁剪目标：

$$L^{\text{clip}}(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

这一项负责训练 Actor，也就是策略网络。

优势为正的动作，概率可以提高。

优势为负的动作，概率可以降低。

但是新旧策略不能差太远。

PPO 通常还需要一个价值网络。价值网络输入状态 s_t ，输出这个状态的价值估计：

$$V_{\phi}(s_t)$$

其中 ϕ 是价值网络参数。真实的目标回报可以记作 V_{target} ，那么价值函数损失可以写成 $(V_{\phi}(s_t) - V_{\text{target}})^2$ ，这和监督学习中的均方误差很像。

价值网络的任务是尽量准确估计一个状态未来能得到多少回报。

为什么需要价值网络？

因为 PPO 需要优势函数 A_t 。优势函数通常可以通过 $A_t = V_{\text{target}} - V_{\phi}(s_t)$ 来估计。

如果价值网络估计得更准，那么优势函数也更可靠，策略更新就更稳定。

如果策略太快变得确定，智能体可能会过早放弃探索。

例如在某个状态下，策略很快变成：

动作 1 的概率是 0.99

动作 2 的概率是 0.01

动作 3 的概率是 0.00

这样智能体几乎只会选择动作 1。

但动作 1 不一定真的是最优的，可能只是前期偶然表现不错。

所以 PPO 通常会加入熵奖励。

熵可以理解为策略的不确定性。

策略越平均，熵越大。

策略越确定，熵越小。

加入熵奖励的目的就是鼓励智能体在训练早期多探索，不要太快锁死在某个动作上。

现在我们把 PPO 的完整训练流程按顺序讲一遍。

第一步，初始化策略网络 Actor 和价值网络 Critic。策略网络负责输出动作概率。价值网络负责输出状态价值。

第二步，用当前策略 π_{old} 和环境交互，采集一批轨迹数据。这些数据包括状态 s_t ；动作 a_t ；奖励 r_t ；下一状态 s_{t+1} ；旧策略下动作概率 $\pi_{\text{old}}(a_t | s_t)$

第三步，根据采集到的奖励，计算每一步的回报。可以用折扣回报，也可以用更常见的 GAE 方法估计优势。

第四步，用价值网络估计每个状态的价值 $V(s_t)$ 。

第五步，计算优势 A_t

第六步，开始多轮更新策略网络。对于每个样本，计算新策略下动作概率 $\pi_{\theta}(a_t | s_t)$ ；然后计算新旧策略概率比 $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$ ；接着代入 PPO 裁剪目标：

$$L^{\text{clip}}(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

第七步，同时更新价值网络。让价值网络输出 $V_{\phi}(s_t)$ 尽量接近目标回报。

第八步，加入熵奖励，防止策略过早变得太确定。

第九步，更新完成后，把当前策略变成新的旧策略，然后继续采集下一批数据。整个过程不断循环。

普通策略梯度通常采集一批数据后，只更新一次或者很少几次。因为策略一旦变化太大，这批旧数据就不可靠了。但是 PPO 有裁剪机制。限制 $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$ 让新策略不会离旧策略太远。因此 PPO 可以对同一批采样数据进行多轮小批量训练，提高了数据利用率，是 PPO 实际效果好的原因之一。

实际 PPO 中，优势函数通常不会简单使用 $A_t = V_{\text{target}} - V_{\phi}(s_t)$ 因为这样方差可能比较大。更常见的是使用 GAE。GAE 的全称是 Generalized Advantage Estimation，中文可以叫“广义优势估计”。

GAE 结合多步 TD 误差来估计优势，平衡偏差和方差。先定义 TD 误差：

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

这个 δ_t 表示当前这一步得到的奖励，加上下一个状态的价值，是否比当前价值网络原本预测的更好。

如果 δ_t 大于 0，说明结果比预期好。

如果 δ_t 小于 0，说明结果比预期差。

GAE 会把多个时间步的 TD 误差加权累积起来：

$$A_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

这里 λ 是一个控制参数。

如果 λ 接近 0，优势估计更依赖一步 TD，方差小但偏差大。

如果 λ 接近 1，优势估计更接近完整回报，偏差小但方差大。

所以 GAE 的作用是在稳定性和准确性之间做平衡。

现在很多大语言模型对齐训练中也会用到 PPO。原因是 RLHF 中的模型也可以看成一个策略。对于语言模型来说：状态可以理解为当前上下文；动作可以理解为下一个 token；策略就是语言模型输出下一个 token 的概率分布；奖励来自奖励模型。比如用户输入一个问题，语言模型生成回答，这一整段回答可以看作一条轨迹，奖励模型给这个回答打分，PPO 要做的就是提高高奖励回答中 token 的生成概率，降低低奖励回答中 token 的生成概率，但是语言模型非常大，如果策略更新太猛，模型可能会崩。例如：语言变得混乱；输出重复；为了骗奖励模型而生成奇怪内容；偏离原来的语言能力。

所以 PPO 通过限制新旧策略差距，让模型在提高奖励的同时，不要离原模型太远。这就是 PPO 在 RLHF 中重要的原因。